

Lappeenranta teknillinen yliopisto
School of Business and Management

Software Development Skills

Diako Jalal, 2524490

LEARNING DIARY, MOBILE MODULE

LEARNING DIARY

22.06.2026

I learned about,

the general information of the course and what the mobile module expects from the project. I chose this module because I use a food and habit tracker on my own phone almost every day, and I wanted to understand what is actually happening behind a screen I already know well. Reading the requirements, several activities, an intent, a broadcast receiver, a list with a custom row layout, and data that survives closing the app, they look like a checklist, so I sketched an app called DayFuel that would need all of those things naturally instead of bolting features on afterwards to tick boxes.

installing Android Studio and getting the default app running on an emulator. The part that did not go smoothly was the emulator, which refused to start and only reported exit code 1. The real cause was hardware virtualisation being switched off in the BIOS, which nothing in the error message suggests. The first download also took closer to forty minutes than the five I had budgeted, because the SDK and one system image are several gigabytes between them.

that I have no version control yet, so I have to back up the project folder myself at the end of every session. I decided to zip the folder with the date in the filename until I set Git up properly in the last phase. It is a crude substitute for an undo button, but working for weeks with no way back at all seems like the one shortcut with real consequences.

24.06.2026

I learned about,

the Kotlin basics the module videos assume I already know `val` and `var`, null safety with `?` and `?::`, and string templates. What was genuinely new to me was that nullability is part of the type itself. In Python a variable can simply be `None` and I find out at runtime; in Kotlin the compiler will not let me touch a nullable value until I have said what should happen when it is null.

how Kotlin differs from the languages I have used before. I found the elvis operator surprising, because I expected to dislike it as unnecessary syntax and instead it replaced a three-line `if (x == null)` block with something I could read at a glance. I also caught myself reaching for `!!` every time the compiler complained, which I now understand is not solving the problem, it switches the safety off and moves the crash to a worse place. String templates were the small pleasure: `"$name has $calories kcal"` instead of concatenating four fragments.

26.06.2026

I learned about,

Kotlin collections and classes, `List` and `MutableList`, data class and enum class. A data class gives me `equals`, `hashCode`, `toString` and `copy` without writing any of them, which is exactly what I want for something like a food entry that is mostly just fields. An enum class for the meal type means the compiler can tell me when I have forgotten a case, rather than the app quietly doing nothing.

how it fits together, by writing a small data class for a food item, putting five of them in a list and totalling the calories in a loop. The mistake I made was declaring the collection as `List` and then trying to call `add` on it. `List` is read-only in Kotlin, which is not what the name suggests to me coming from Python — I needed `MutableList`. I also got a percentage that stubbornly printed 0 because I had written `value / goal * 100` with integers: the division

truncates to 0 before the multiplication ever runs, so the whole expression collapses. Multiplying first fixes it, and I will not make that mistake again.

29.06.2026

I learned about,

the structure of an Android project — the manifest, the res folder, and why there are two build.gradle files. One belongs to the whole project and one to the module, and almost everything I actually need to change lives in the module one. The manifest turned out to be the file I would later forget at exactly the wrong moment.

what Gradle sync actually does, how to create an emulator in Device Manager, and how to read Logcat. Logcat unfiltered is thousands of lines of system chatter, so filtering it to my own package and to Error level is what made it usable. The "app keeps stopping" dialog tells me nothing at all; Logcat tells me the exact line. I also learned to check the Build panel rather than trusting the Run button, which is greyed out with no explanation whenever a sync has failed.

XML layouts and connecting them to code with findViewById and setOnClickListener. I built a small counter app to check I understood it. layout_weight did nothing until I realised the weighted dimension has to be 0dp rather than wrap_content — weight divides the leftover space, so asking for the content's width first defeats the point. I also had a screen with its bottom cut off, which was simply content taller than the display with no ScrollView around it.

01.07.2026

I learned about,

the activity lifecycle, and specifically why onResume is the right place to refresh what is on screen. I had put my refresh call in onCreate, so the dashboard still showed the old totals

after I came back from adding food. onCreate does not run again when an activity is merely returned to. This is the cleanest possible demonstration of why the lifecycle exists, and it stopped being something I had memorised and started being something I understood.

explicit intents and opening a second activity: `Intent(this, AddEntryActivity::class.java)` followed by `startActivity`, with `putExtra` and `getStringExtra` for passing values across. `getStringExtra` returned null the first time because I had typed the key slightly differently in the two places, so I moved the key into a constant and stopped writing it as a loose string.

that an activity has to be declared in `AndroidManifest.xml` or the app crashes the moment you try to open it. That cost me the best part of an hour, because the crash happens when you tap the button and so it looks like a problem with the button, the click listener, or the layout, anything except a file I had not touched. The stack trace named `ActivityNotFoundException`, and searching for the exception name rather than describing the symptom found the answer in one result.

03.07.2026

I learned about,

implicit intents, and how sharing text with `ACTION_SEND` lets another app handle it. I do not have to know which apps can receive it, `Intent.createChooser` asks the system and shows whatever the user has installed. This is the part of Android that finally made the design click for me: apps advertise what they are able to do, and the system does the matching.

what a `BroadcastReceiver` is, and the difference between declaring one in the manifest and registering it in code. A manifest-declared receiver can be woken up when my app is not running, which is exactly what a reminder needs; one registered at runtime has to be unregistered in the matching lifecycle method, and if it is not, it leaks and then crashes on the first rotation. I also learned that a `BOOT_COMPLETED` receiver never fires unless the app has been launched at least once since install.

06.07.2026

I learned about,

AlarmManager and PendingIntent, and how to make something happen later without the app running at all. PendingIntent needs FLAG_IMMUTABLE on Android 12 and newer or it throws IllegalArgumentException straight away, at least that is an error that names its own cause.

Notifications, notification channels, and the POST_NOTIFICATIONS permission newer Android versions ask for at runtime. The problem that took me longest was a notification that simply never appeared, with nothing in Logcat and no error anywhere. It turns out three completely separate faults produce exactly the same silence: the channel was never created, the channel ID in the builder did not match the one I had created, and on Android 13 the permission had been denied. Worse, channel settings are frozen after the channel is first created, so when I changed the importance nothing happened until I uninstalled the app entirely and let it create the channel again. I lost an evening to that and would not have found it by reading my own code, because my own code was correct.

that IntentService from the video is deprecated now and WorkManager has replaced it. The videos are several years old and this is not the only place where the platform has moved on, so I have started checking the official Android guides for anything API-related rather than trusting either the video or the top search result, several of the Stack Overflow answers I found were written in 2014.

08.07.2026

I learned about,

ListView and ArrayAdapter, starting with a plain adapter over a list of strings just to get something on screen. Doing the crude version first turned out to be a good habit: when the list came up blank I already knew the data was fine and the problem had to be the adapter.

how to write my own adapter with a custom row layout, overriding getView. What confused me about convertView was that I assumed I was handed a fresh row every time. I set the meal icon only inside the if (convertView == null) branch, and as soon as the list was long enough to scroll, rows came back carrying the previous row's icon — breakfast items showing a dinner icon. Rows are recycled to save work, so every field has to be set on every call without exception. That bug taught me more about how ListView works than the explanation did.

10.07.2026

I learned about,

ImageView, drawables, and how to add icons with Vector Asset Studio. Right-clicking res → New → Vector Asset gives me Material icons as XML that scale to any density, instead of exporting five sizes of the same PNG. One of my icons was invisible for a good ten minutes — white fill on a white background, not anything wrong with the file.

shape drawables, and how to make the coloured circles sitting behind the meal icons. They are a few lines of XML rather than an image, which keeps the app small and lets me reuse one shape with different colours.

handling long presses on list rows with setOnItemLongClickListener. It has to return true, otherwise Android treats the press as unhandled and fires the ordinary click as well, so long-pressing an entry both opened it and offered to delete it. Resource naming also caught me out: ic-breakfast.xml and icBreakfast.xml both fail to build, because resource filenames allow only lowercase letters, digits and underscores.

13.07.2026

I learned about,

SharedPreferences, and why it can only save single values and not a list of objects. Forgetting `apply()` on the editor means the value is simply gone at the next launch, with no error at any point to suggest anything went wrong.

converting my food entries to JSON text with `JSONObject` and `JSONArray` so they can be stored as a single string, and reading them back into objects. I switched to `optString` and `optInt` instead of `getString` and `getInt`, because a key that is missing throws a `JSONException` rather than returning nothing — which matters for entries I had already saved before adding a field.

`SimpleDateFormat`, and using a `yyyy-MM-dd` string as the key so every day gets its own slot, with a separate dates key listing every day that has data. I also found a real bug in my own code here: I was building the storage key with `Locale.getDefault()`, so on a device set to a locale that formats numbers differently the key changes shape and the previous day's entries become unreachable. Storage keys should use `Locale.US` and only text shown to the user should follow the device locale. It is one line, it would never have failed on my machine, and finding it myself was the most satisfying moment of the project so far.

15.07.2026

I learned about,

how much easier it is to plan the data before writing any code. I decided on paper what an entry is — name, calories, meal type and a timestamp — what identifies it, and how I would find everything logged today. The timestamp mattered more than I expected: it is what lets me delete the right entry after the list has been sorted, instead of deleting by position and removing something else.

building the dashboard and the add food screen. I ran the app after every small piece rather than writing twenty files and then pressing Run, which meant that when something broke I knew exactly which change had caused it. The first time I tried it the other way round I got a screen full of errors with no idea where to start.

why it is worth putting all the saving code in one DataManager class instead of scattering it across activities. Two activities calling `getSharedPreferences` with slightly different names look identical when you read them and produce data that mysteriously does not appear on the other screen. With one class there is exactly one place the preference name is written, and the bug cannot happen.

17.07.2026

I learned about,

building the food log screen with my own adapter, and deleting entries with an `AlertDialog` to confirm first. The list refused to update after I added an item until I understood that the adapter has no way of knowing the underlying data changed — `notifyDataSetChanged()` exists for precisely that, and it is not something the compiler can warn me about.

input validation and showing errors with a `Toast`. The case I had not thought about was letters typed into the calorie field: `toInt()` throws and takes the whole app down, while `toIntOrNull()` returns null and lets me show a message instead. An empty name and an empty number field were the other two I had been quietly avoiding, and both were one line away from crashing during the demo.

the goals screen, and using a `SeekBar` instead of a text field. It removes a whole category of invalid input, because nobody can type nonsense into something that only slides between two numbers. Choosing the right control turns out to be a cheaper fix than validating a free-text field.

20.07.2026

I learned about,

finishing the water reminders. This was the hardest part of the project because almost nothing about it gives feedback: `setInexactRepeating` is deliberately imprecise and Doze delays it further, so sitting and waiting to see whether it fired told me nothing either way. I ended up broadcasting to the receiver manually to prove the notification path worked, and only then trusting the alarm around it. Splitting a problem into a part I can test quickly and a part I cannot is a technique I will reuse.

styling the app with `colors.xml` and `themes.xml`, and using `MaterialCardView` for the dashboard sections. Defining the colours in one file and referring to them everywhere else meant I could change the whole palette in a minute instead of hunting through six layout files, which is the same lesson as the `DataManager` one in a different costume.

testing the cases I had been avoiding — empty fields, a day with nothing logged, deleting the last entry. What broke was the day with no entries: my average calculation divided by the number of entries and crashed on zero. I also spent twenty minutes convinced my Kotlin was broken because every reference to `R` was red, when the real cause was a missing `>` in a layout file I had edited an hour earlier. One broken XML file kills the whole `R` class, and the editor points confidently at the wrong place. I now check the Build panel before I believe a single red squiggle.

22.07.2026

I learned about,

writing a README that someone who has never seen the project can follow. My first attempt said "open it in Android Studio and run it", which quietly assumes my exact setup. The version I kept states the minimum SDK, says it needs an emulator or device on Android 8 or newer, and explains where to allow the notification permission — because without it the reminders silently do nothing and the app looks broken.

recording the demo video of the app running. I used the screen-record button in Device Manager rather than setting up OBS, which is far simpler for an emulator. I tested the

unhappy paths before recording rather than after, and checked the required features against the brief before I stopped the recording, which still did not save me from re-recording twice.

26.07.2026

I learned about,

setting up Git and putting the finished project in a repository. I left version control until the end, which meant every Git problem I was ever going to have arrived on the same evening: "Author identity unknown" on the very first commit because user.name and user.email were not set, and then a first commit stuffed with build output because I had added .gitignore afterwards instead of before.

what .gitignore is for, and why build/, .gradle/, .idea/ and local.properties must never be committed. They are either regenerated on every build or specific to my machine — local.properties in particular holds the SDK path, so committing it points a fresh clone at a folder that exists on no other computer. Adding .gitignore late is also not enough on its own, because Git keeps tracking whatever it already tracks; I had to run `git rm -r --cached` on the folders that were already in before they would disappear from git status.

how to split the work into a few sensible commits instead of one enormous one — project setup, then layouts, then storage, then reminders, then styling. It follows the order I actually built things in and is far easier for anyone else to read than a single commit called "everything". I also verified the result by cloning the repository into a new folder and opening it, which is the only way to find out whether I committed something machine-specific or left something out.

what I would do differently next time about version control, now that I have seen the difference between a history that shows progress and one that does not. Mine is a handful of commits made on a single evening: honest about the order I worked in, but it tells nobody anything about the five weeks behind it, and if anyone looks at the history for evidence of steady work it is not there. The cost was not only the history. For the entire project my only

undo was a dated zip file, and I know I avoided experiments I would have tried freely if I could have thrown them away for nothing. Next time the repository goes up before the first line of code, and .gitignore goes in before the first commit.